Lecture 5 Notes: Representation Learning (Part 3)

Data Compression, PCA & Residue Analysis

IIT Madras — Machine Learning Techniques (CS2002)

Where the Approach in the Previous Video is Incomplete

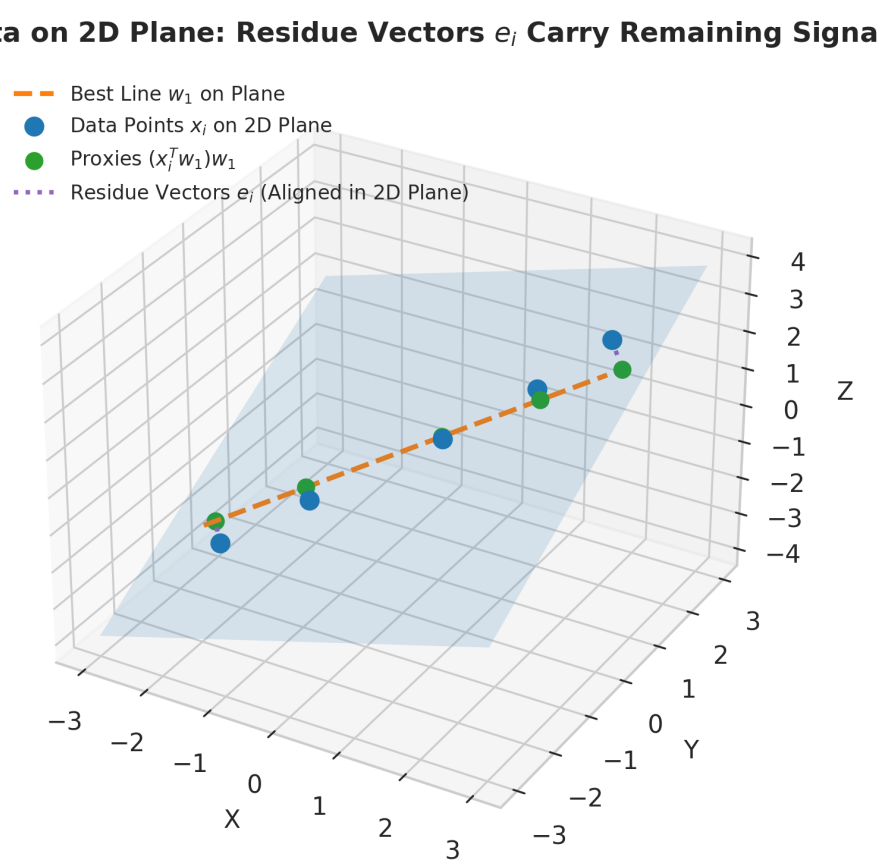
In **Lecture 4**, we derived a method to find the single best line w_1 (first principal component) that minimizes reconstruction error across a dataset. However, that single-line formulation is **incomplete** due to two fundamental limitations:

1. **Single Line vs. Higher-Dimensional Structures:** Real-world data often lives on higher-dimensional subspaces (e.g., a 2D plane in 3D space). Projecting onto a single 1D line discards structured signals in the remaining dimensions. The part discarded as "reconstruction error" is **not random noise** — it contains structured information aligned along secondary directions.
2. **The Origin Constraint & Uncentered Data:** The optimization problem $\max w^T C w$ assumes that projection lines pass through the origin $(0, 0)$. If a dataset is not centered at the origin, forcing lines through $(0, 0)$ results in severe reconstruction error and selects directions that do not reflect the true spread of the data.

1 1. The Limitations of Single-Line Projection

In Lecture 4, we assumed that finding a single vector $w_1 \in \mathbb{R}^d$ with $\|w_1\|^2 = 1$ is sufficient to represent a dataset $\{x_1, x_2, \ldots, x_n\}$.

Consider moving from 2D space to 3D space. Suppose our dataset consists of vectors in $\mathbb{R}^3$ that lie along a **two-dimensional plane** inside the 3D space.



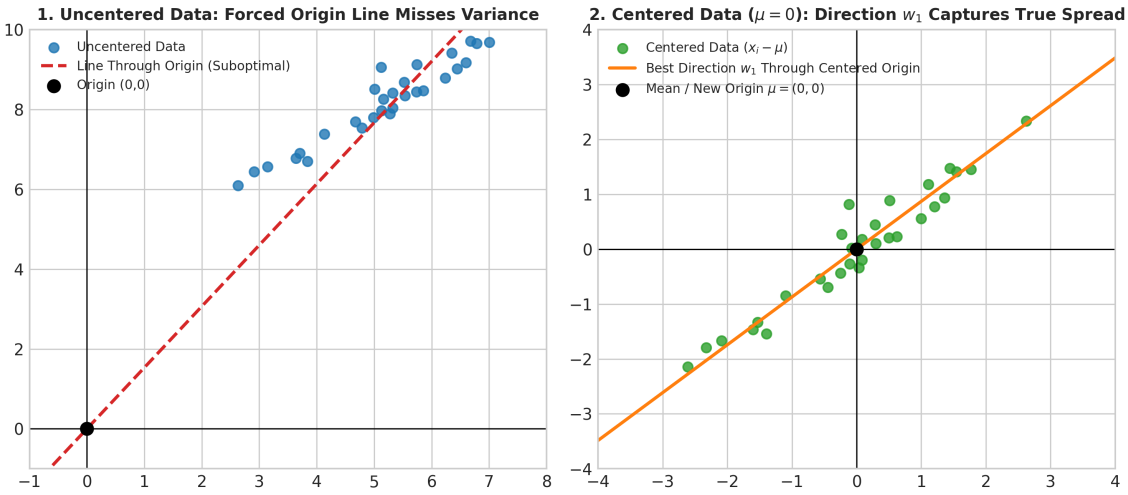
When we apply our single-line compression algorithm to this 3D dataset:

- The algorithm finds a single best line w_1 that lies within the 2D plane.
- All data points x_i are projected onto w_1 as proxies: $\hat{x}_i = (x_i^T w_1)w_1$.
- The **error/residue vector** for each point is $e_i = x_i - (x_i^T w_1)w_1$.

Key Insight: If we plot the error vectors $e_1, e_2, \ldots, e_n$ in 3D space, we observe that they are **not scattered randomly**. Instead, all error vectors align neatly along a **second direction** within the same 2D plane! This proves that the residue contains structured, informative signal rather than noise.

2 2. Resolving Data Offset: The Necessity of Mean Centering

Before attempting to extract sequential projection directions, we must resolve the uncentered data issue.



- **The Issue:** Our representation models lines passing through the origin ($y = c \cdot w$). If raw data points are situated far from the origin, a line passing through $(0, 0)$ will suffer large error and pick an incorrect orientation.
- **The Fix (Mean Centering):**
  1. Compute the sample mean vector $\mu \in \mathbb{R}^d$:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

  2. Subtract μ from every data point to create the centered dataset $\{\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_n\}$:

$$\tilde{x}_i = x_i - \mu \quad \text{for all } i = 1, \ldots, n$$

  The centered dataset has a mean of zero ($\frac{1}{n} \sum \tilde{x}_i = 0$), guaranteeing that lines passing through the origin properly align with the dataset's natural variance.

3 3. The Iterative Residue Analysis Algorithm

To extract all underlying structural directions without discarding informative signal, we use an **Iterative Residue Extraction Procedure**:

1. **Step 0: Center the Data.**

$$\tilde{x}_i = x_i - \mu, \quad \text{where } \mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

2. **Step 1: Find First Direction (w_1).** Solve the Lecture 4 optimization problem on centered data:

$$\max_{w_1 : \|w_1\|^2 = 1} w_1^T C w_1 \quad \text{where } C = \frac{1}{n} \sum_{i=1}^{n} \tilde{x}_i \tilde{x}_i^T$$

3. **Step 2: Compute the Residue Dataset.** Subtract the projection along w_1 to form the first residue dataset $\{e_1^{(1)}, e_2^{(1)}, \ldots, e_n^{(1)}\}$:

$$e_i^{(1)} = \tilde{x}_i - (\tilde{x}_i^T w_1) w_1$$

4. **Step 3: Find Second Direction (w_2).** Treat $\{e_i^{(1)}\}$ as a brand-new dataset and find the best direction w_2 that maximizes variance on these residue vectors.

5. **Step 4: Multi-Stage Extension.** Repeat the residue calculation and direction extraction iteratively for $w_3, w_4, \ldots, w_k$ up to dimension d.

Open Questions Left by the Professor at the End of Lecture 5

At the conclusion of Lecture 5, Professor Arun Rajkumar leaves us with **four critical questions** that form the bridge to the complete Principal Component Analysis (PCA) algorithm:

1. **How to solve** $\max_{w:\|w\|^2=1} w^T C w$**?** How do we formally derive and compute the eigenvector solution using linear algebra?

2. **How many times should we repeat the procedure?** How many principal components / residue iterations ($k \leq d$) should we retain to balance compression and reconstruction accuracy?

3. **Where exactly is the compression happening now in the algorithm?** What scalar codes are stored on disk versus what direction vectors are preserved?

4. **What are the representations that we are really learning using this?** What do the projection coefficients $c_{i,1}, c_{i,2}, \ldots, c_{i,k}$ and direction vectors $w_1, w_2, \ldots, w_k$ represent geometrically and structurally?

Answering these four questions in the next lecture gives us our first major data science algorithm: ***Unsupervised Representation Learning via PCA****.*